

Assignment 2: Odometry Robot P3DX

Author(s): Muhammad Fairuzzaki Setyawan
Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name: Sistem Robot Otonom
YouTube video link: <https://youtu.be/wRoeHtfb2H4>
GitHub link: <https://github.com/fairuzzaki7/plot-temporal-joint-body-velocity-sro/tree/main>

1. Penjelasan Awal

Tugas ini merupakan tugas untuk menentukan posisi dari robot *pioneer* P3DX dengan menggunakan dua metode, dimana metode pertama menggunakan *theta* dari sensor internal dan metode kedua yaitu menggunakan orientasi *gamma* yang berasal dari hasil integral kecepatan sudut. Selain itu tugas ini memiliki fungsi untuk menampilkan plot spasial posisi robot *odometry* yang dimana dengan membandingkan posisi diantara penggunaan kedua metode tersebut.

2. Kode Program

Berikut adalah program untuk mensimulasikan robot *pioneer* P3DX pada CoppeliaSim, menampilkan posisi dan kecepatan robot pada masing-masing sumbu dengan menggunakan metode *theta* dan metode *gamma*, menampilkan nilai orientasi robot berdasarkan hasil integral kecepatan sudut dan sensor, serta menampilkan plot spasial dari posisi robot *pioneer* P3DX yang berfungsi untuk membandingkan penggunaan kedua metode tersebut. Program ini menggunakan bahasa python yang terhubung langsung dengan aplikasi CoppeliaSim:

```
1  # %%
2  # %%
3  import time
4  import math
5  import numpy as np
6  import matplotlib.pyplot as plt
7  from datetime import datetime
8  from coppeliasim_zmqremoteapi_client import RemoteAPIClient
9
10 # %%
11 # 1. Setup Connection
12 client = RemoteAPIClient()
13 sim = client.require('sim')
14
15 # %%
16 # 2. Start Simulation
17 sim.startSimulation()
18 print("Simulation Started")
19
20 # %%
21 # 3. Simple Test: Post a message to CoppeliaSim status bar
22 sim.addLog(1, "Hello from Python!")
23
24 p3dx_RW = sim.getObject("/PioneerP3DX/rightMotor")
25 p3dx_LW = sim.getObject("/PioneerP3DX/leftMotor")
26 p3dx = sim.getObject("/PioneerP3DX")
27
28 rw = 0.195/2
29 rb = 0.381/2
30 d = 0.05
31 dt = 0.001
32 x_pose_int = 0
```

```

33 y_pose_int = 0
34 x_pose_int2 = 0
35 y_pose_int2 = 0
36 gamma_int = 0
37
38 # Data storage for temporal plot
39 time_data = []
40 x_pose = []
41 y_pose = []
42 x_pose2 = []
43 y_pose2 = []
44
45 # %%
46 try:
47     # 4. Main Loop (Run for 20 seconds)
48     start_time = time.time()
49     elapsed_prev = 0
50
51     while (time.time() - start_time) < 20:
52
53         # --- STUDENT CODE GOES HERE ---
54         # Example: Print elapsed time
55         elapsed = time.time() - start_time
56         print(f"Running... {elapsed:.1f}s", end="\r")
57
58         dt = elapsed - elapsed_prev
59         elapsed_prev = elapsed
60
61
62         wr_vel = sim.getJointTargetVelocity(p3dx_RW)
63         wl_vel = sim.getJointTargetVelocity(p3dx_LW)
64
65         sim.addLog(1, f"RW:{wr_vel:.1f}rad/s, LW:{wl_vel:.1f}rad/s")
66
67         vx = (wr_vel + wl_vel)*rw/2
68         wx = (wr_vel - wl_vel)*rw/rb
69
70
71         theta = sim.getObjectOrientation(p3dx, sim.handle_world)
72
73         x_dot = vx * math.cos(theta[2])
74         y_dot = vx * math.sin(theta[2])
75
76         x_pose_int = x_pose_int + x_dot * dt
77         y_pose_int = y_pose_int + y_dot * dt
78
79         gamma_int = gamma_int + wx * dt
80
81         x_dot2 = vx * math.cos(gamma_int)
82         y_dot2 = vx * math.sin(gamma_int)
83
84         x_pose_int2 = x_pose_int2 + x_dot2 * dt
85         y_pose_int2 = y_pose_int2 + y_dot2 * dt
86
87         sim.addLog(1, f"RW:{wr_vel:.1f}rad/s, LW:{wl_vel:.1f}rad/s")
88         sim.addLog(1, f"vx:{vx:.1f}m/s, wx:{wx:.1f}rad/s")
89         sim.addLog(1, f"x_dot:{x_dot:.1f}m/s, y_dot:{y_dot:.1f}m/s")
90         sim.addLog(1, f"x_dot2:{x_dot2:.1f}m/s, y_dot2:{y_dot2:.1f}m/s")
91         sim.addLog(1, f"gamma_int:{gamma_int:.2f}rad")
92         sim.addLog(1, f"theta:{theta[2]:.2f}rad")
93         sim.addLog(1, f"x_pose_int:{x_pose_int:.1f}m, y_pose_int:{y_pose_int:.1f}m")
94         sim.addLog(1, f"x_pose_int2:{x_pose_int2:.1f}m, y_pose_int2:{y_pose_int2:.1f}m")

```

```

95
96
97
98
99     # Save data for plotting
100    time_data.append(elapsed)
101    x_pose.append(x_pose_int)
102    y_pose.append(y_pose_int)
103    x_pose2.append(x_pose_int2)
104    y_pose2.append(y_pose_int2)
105
106
107
108
109    finally:
110        # 5. Stop Simulation safely
111        sim.stopSimulation()
112        print("\nSimulation Stopped")
113
114        # Temporal Plot
115    plt.figure()
116    plt.plot(x_pose, y_pose, label="From Orientation")
117    plt.plot(x_pose2, y_pose2, '--', label="From Integration")
118    plt.xlabel("X Position (m)")
119    plt.ylabel("Y Position (m)")
120    plt.title("Robot Path")
121    plt.legend()
122    plt.grid(True)
123    plt.axis('equal')
124    plt.tight_layout()
125    plt.show()

```

1. Import Library

- "Import RemoteAPIClient": Berfungsi untuk menghubungkan Python menuju aplikasi CoppeliaSim
- "Import time" dan "Import numpy": Berfungsi untuk menjalankan *timer* serta perhitungan secara numerik
- "Import matplotlib": Berfungsi untuk menjalankan plottingan pada grafik

2. Menghubungkan ke CoppeliaSim

- "client = RemoteAPIClient()": Membuat hubungan antara python dengan CoppeliaSim
- "sim = client.require('sim')": Handle utama diambil sebagai awal pengaksesan fungsi pada CoppeliaSim

3. Menjalankan dan Memberhentikan Simulasi

- "Sim.StartSimulation()": Berfungsi untuk menjalankan simulasi dari Python
- "sim.stopSimulation()": Berfungsi untuk memberhentikan simulasi dari Python

4. Menampilkan Pesan pada CoppeliaSim

- "sim.addlog(1, "Hello from Python!")"
- "sim.addLog(1, f"RW:wr_vel:.1frad/s, LW:wl_vel:.1frad/s")"
- "sim.addLog(1, f"Vx:vx:.1fm/s, WX:wx:.1frad/s")"
- "sim.addLog(1, f"x_dot:x_dot:.1fm/s, y_dot:y_dot:.1fm/s")"
- "sim.addLog(1, f"x_dot2:x_dot2:.1fm/s, y_dot2:y_dot2:.1fm/s")"
- "sim.addLog(1, f"gamma_int:gamma_int:.2frad"

- `"sim.addLog(1, f"theta:theta[2]:.2frad")"`
- `"sim.addLog(1, f"x_pose_int:x_pose_int:.1fm, y_pose_int:y_pose_int:.1fm")"`
- `"sim.addLog(1, f"x_pose_int2:x_pose_int2:.1fm, y_pose_int2:y_pose_int2:.1fm")"`

Berfungsi untuk mengirim pesan ke CoppeliaSim, sehingga pada terminal CoppeliaSim dapat tertuliskan sesuai yang diminta pada Python

5. Mengambil Handle Motor

- `"p3dx_RW = sim.getObject("/PioneerP3DX/rightMotor")"`
- `"p3dx_LW = sim.getObject("/PioneerP3DX/leftMotor")"`
- `"p3dx = sim.getObject("/PioneerP3DX")"`

Berperan untuk mengambil alih *handle* motor baik pada bagian kiri dan bagian kanan.

0.0.1 6. Parameter Nilai Robot

- `"rw = 0.195/2"`: Jari-jari roda robot P3DX
- `"rb = 0.381/2"`: Jarak roda kanan atau kiri dengan pusat robot
- `"dt = 0.001"`: Nilai awal *time step* sebelum waktu dimulai
- `"x_pose_int = 0, y_pose_int = 0, x_pose_int2 = 0, y_pose_int2 = 0"`: Posisi awal robot pada kedua metode
- `"gamma_int = 0"`: Nilai awal orientasi robot metode kedua

6. Waktu pada Sistem

- `"while (time.time() - start_time) <20:"`
- `"elapsed = time.time() - start_time"`
- `"dt = elapsed - elapsed_prev"`

Berfungsi untuk memberikan lama waktu dari sistem berjalan, memberikan perintah untuk menghitung ketika sistem mulai berjalan, serta menghitung selisih waktu antara iterasi sekarang dengan sebelumnya.

7. Kecepatan Robot

- `"x_dot = vx * math.cos(theta[2])"`
- `"y_dot = vx * math.sin(theta[2])"`

Berfungsi untuk mendapatkan kecepatan robot pada masing-masing sumbu dengan menggunakan metode pertama.

- `"x_dot2 = vx * math.cos(gamma_int)"`
- `"y_dot2 = vx * math.sin(gamma_int)"`

Berfungsi untuk mendapatkan kecepatan robot pada masing-masing sumbu dengan menggunakan metode kedua.

8. Posisi Robot

- `"x_pose_int = x_pose_int + x_dot * dt"`
- `"y_pose_int = y_pose_int + y_dot * dt"`

Berfungsi untuk mendapatkan posisi robot pada masing-masing sumbu dengan menggunakan metode pertama.

- `"x_pose_int2 = x_pose_int2 + x_dot2 * dt"`
- `"y_pose_int2 = y_pose_int2 + y_dot2 * dt"`

Berfungsi untuk mendapatkan posisi robot pada masing-masing sumbu dengan menggunakan metode kedua.

9. Penyimpanan Data

- `time_data.append(elapsed)`
- `x_pose.append(x_pose_int)`
- `y_pose.append(y_pose_int)`
- `x_pose2.append(x_pose_int2)`
- `y_pose2.append(y_pose_int2)`

Berperan untuk menyimpan data dari posisi masing-masing sumbu dengan penggunaan masing-masing metode yang nantinya akan ditampilkan pada grafik plot spasial.

10. Plot Grafik

- `"plt.subplot()"`
- `"plt.plot()"`
- `"plt.show()"`

Berfungsi untuk menentukan tampilan plot grafik serta menampilkan pada layar.

3. Hasil Plot Temporal dan Pembahasan

Berikut merupakan plot spasial dari posisi robot *pioneer* P3DX dimana membandingkan dua metode orientasi yang ditampilkan dalam masing-masing sumbu:

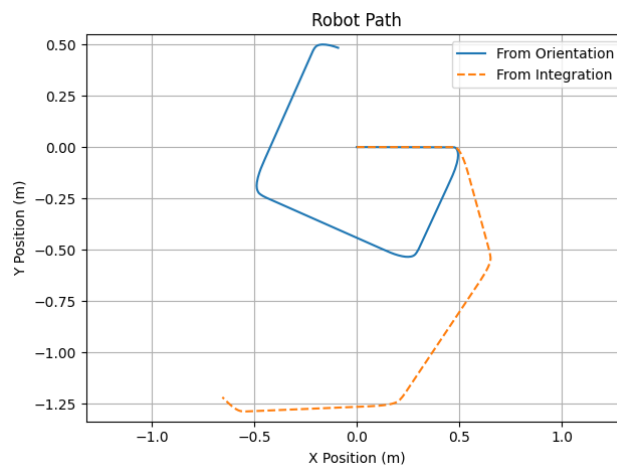


Figure 1: Perbandingan Plot Spasial Posisi Robot P3DX dengan Metode integral dan Sensor Internal

Pembahasan Grafik

Plot tersebut adalah plot spasial posisi robot *pioneer* P3DX berdasarkan sumbu-X dan sumbu-Y dengan menggunakan orientasi *theta* yaitu melalui sensor internal robot yang ditandai dengan garis biru dan orientasi *gamma* yaitu melalui hasil integral kecepatan sudut pada robot yang ditandai dengan garis oranye putus-putus. Dari hasil plot dapat dianalisis bahwa awal jalan robot diantara kedua metode sama hingga belokan pertama. Setelah belokan pertama, jalur diantara kedua metode mulai berbeda dimana penggunaan sensor internal lebih akurat dan hasil perhitungan integral sedikit melenceng yang dapat dikatakan mengalami kondisi *drift* yang diakibatkan adanya akumulasi error dari hasil perhitungan integral.